Experiment 8: Two-Layer Neural Network with Backpropagation

Question

Problem Statement: The bank's screener from Experiment 7 fails on non-linear cases. The data team needs a hidden layer and must understand how errors flow back. The XOR-style loan dataset and a two-moons customer-segmentation dataset. The objective is to build a 2-layer network from scratch and implement backpropagation manually. Use NumPy, Matplotlib.

Tasks:

1. Initialise weights for an input–hidden–output network in NumPy.
2. Write the forward pass with sigmoid activations.
3. Derive and code the backward pass using the chain rule.
4. Train with gradient descent and plot loss against epochs.
5. Plot the non-linear decision boundary; verify gradients numerically for one weight.

Code

```
"""Experiment 8: Two-Layer Neural Network with Backpropagation (from scratch)"""
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons


np.random.seed(0)


# ---------------- Datasets ----------------
# (a) XOR-style loan data: approved when credit score and income ratio are BOTH high or BOTH low
X_xor = np.random.uniform(-1, 1, (300, 2))
y_xor = ((X_xor[:, 0] * X_xor[:, 1]) > 0).astype(float).reshape(-1, 1)
# (b) two-moons customer segmentation
X_moon, y_moon = make_moons(300, noise=0.2, random_state=0)
y_moon = y_moon.reshape(-1, 1).astype(float)


# ---------------- Task 1: initialise weights ----------------
def init_params(n_in, n_hidden, n_out, seed=0):
    rng = np.random.RandomState(seed)
    return {"W1": rng.randn(n_in, n_hidden) * 0.5, "b1": np.zeros((1, n_hidden)),
            "W2": rng.randn(n_hidden, n_out) * 0.5, "b2": np.zeros((1, n_out))}


# ---------------- Task 2: forward pass ----------------
def sigmoid(z):
    return 1 / (1 + np.exp(-z))


def forward(X, p):
    z1 = X @ p["W1"] + p["b1"]
    a1 = sigmoid(z1)                      # hidden layer
    z2 = a1 @ p["W2"] + p["b2"]
    a2 = sigmoid(z2)                      # output layer
    return z1, a1, z2, a2


def bce_loss(y, a2):
    eps = 1e-9
    return -np.mean(y * np.log(a2 + eps) + (1 - y) * np.log(1 - a2 + eps))


# ---------------- Task 3: backward pass (chain rule) ----------------
# L = BCE(a2, y);  dL/dz2 = a2 - y  (BCE + sigmoid simplification)
# dL/dW2 = a1^T dz2 ; dL/da1 = dz2 W2^T ; dL/dz1 = da1 * a1(1-a1) ; dL/dW1 = X^T dz1
def backward(X, y, cache, p):
    z1, a1, z2, a2 = cache
    m = X.shape[0]
    dz2 = (a2 - y) / m
    dW2 = a1.T @ dz2
    db2 = dz2.sum(axis=0, keepdims=True)
    da1 = dz2 @ p["W2"].T
    dz1 = da1 * a1 * (1 - a1)
    dW1 = X.T @ dz1
    db1 = dz1.sum(axis=0, keepdims=True)
```

```
    return {"W1": dW1, "b1": db1, "W2": dW2, "b2": db2}


# ---------------- Task 4: training with gradient descent ----------------
def train(X, y, n_hidden=8, lr=1.0, epochs=5000, seed=0):
    p = init_params(X.shape[1], n_hidden, 1, seed)
    losses = []
    for ep in range(epochs):
        cache = forward(X, p)
        losses.append(bce_loss(y, cache[3]))
        g = backward(X, y, cache, p)
        for k in p:
            p[k] -= lr * g[k]
    return p, losses


def plot_boundary(p, X, y, title, ax):
    xx, yy = np.meshgrid(np.linspace(X[:, 0].min() - 0.5, X[:, 0].max() + 0.5, 300),
                         np.linspace(X[:, 1].min() - 0.5, X[:, 1].max() + 0.5, 300))
    zz = forward(np.c_[xx.ravel(), yy.ravel()], p)[3].reshape(xx.shape)
    ax.contourf(xx, yy, zz, levels=[0, 0.5, 1], alpha=0.3, colors=["red", "green"])
    ax.scatter(X[:, 0], X[:, 1], c=y.ravel(), cmap="RdYlGn", edgecolors="k", s=20)
    ax.set_title(title)


# ---------------- Task 5: gradient check ----------------
def gradient_check(X, y, p, key="W1", idx=(0, 0), eps=1e-5):
    cache = forward(X, p)
    analytic = backward(X, y, cache, p)[key][idx]
    p[key][idx] += eps; lp = bce_loss(y, forward(X, p)[3])
    p[key][idx] -= 2 * eps; lm = bce_loss(y, forward(X, p)[3])
    p[key][idx] += eps
    numeric = (lp - lm) / (2 * eps)
    return analytic, numeric


fig, ax = plt.subplots(2, 2, figsize=(12, 9))
for row, (name, X, y) in enumerate([("XOR-style loan data", X_xor, y_xor), ("Two-moons segmentation", X_moon,
y_moon)]):
    # check gradients on the untrained network first
    p0 = init_params(2, 8, 1)
    a, n = gradient_check(X, y, p0, "W1", (0, 0))
    print(f"[{name}] gradient check W1[0,0]: analytic={a:.8f}  numeric={n:.8f}  diff={abs(a-n):.2e}")
    a, n = gradient_check(X, y, p0, "W2", (0, 0))
    print(f"[{name}] gradient check W2[0,0]: analytic={a:.8f}  numeric={n:.8f}  diff={abs(a-n):.2e}")


    p, losses = train(X, y)
    acc = ((forward(X, p)[3] > 0.5) == y).mean()
    print(f"[{name}] final loss = {losses[-1]:.4f}, accuracy = {acc*100:.1f}%\n")
    ax[row, 0].plot(losses); ax[row, 0].set_title(f"{name}: loss vs epochs")
    ax[row, 0].set_xlabel("Epoch"); ax[row, 0].set_ylabel("BCE loss")
    plot_boundary(p, X, y, f"{name}: decision boundary (acc {acc*100:.1f}%)", ax[row, 1])
plt.tight_layout(); plt.show()
```

Output

[XOR-style loan data] gradient check W1[0,0]: analytic=0.00029659 numeric=0.00029659 diff=1.13e-11

[XOR-style loan data] gradient check W2[0,0]: analytic=-0.02589536 numeric=-0.02589536 diff=1.36e-10

[XOR-style loan data] final loss = 0.0396, accuracy = 100.0%

[Two-moons segmentation] gradient check W1[0,0]: analytic=-0.03827935 numeric=-0.03827935 diff=1.02e-10

[Two-moons segmentation] gradient check W2[0,0]: analytic=-0.10896026 numeric=-0.10896026 diff=3.61e-10

[Two-moons segmentation] final loss = 0.0658, accuracy = 98.0%

